LECTURE 06

Analyzing Non-Recursive Algorithms

RAM model • basic operations • exact counts • best / worst / average case

Now that an algorithm is correct — how much work does it actually do?

INPUT SIZE

What does n measure?

BASIC OP

What repeats most?

COUNT

How many times?

CASES

Best • worst • average

COUNT FIRST

Big-O comes in the next lecture.

CDF OUTCOMES

What should you be able to do by the end?

Lecture 6 follows the CDF topic “Analyzing Non-Recursive Algorithms.”

MODEL

explain the simplified RAM model used for analysis

ANALYZE

identify operations performed by non-recursive algorithms

COUNT

count a basic operation to estimate running time

DISTINGUISH

best-case, worst-case, and average-case behavior

RELATE

connect input characteristics to performance

APPLY

mathematical analysis to representative algorithms

Required CDF example: analyze iterative maximum-finding under different input arrangements.

COURSE MAP

Where Lecture 6 fits

The course now shifts from proving correctness to measuring efficiency.

L3	L4	L5	L6	L7
State correctness	Prove loops	Prove recursion	Count the work	Describe growth

Today: exact / concrete operation counts. Next lecture: Big-O, Big-Ω and Θ growth-rate notation.

BRIDGE

Correct first. Efficient second.

Efficiency comparison is meaningful only among algorithms that solve the required problem correctly.

1 CORRECTNESS

Does the algorithm return the required result for every valid input?

2 ANALYSIS

How many primitive operations does it perform as the input changes?

3 SELECTION

Among correct choices, which solution uses resources more effectively?

A fast wrong answer is still wrong.

Why not just run the program and time it?

Elapsed time is useful experimentally, but it mixes algorithm quality with machine and implementation details.

CLOCK TIME DEPENDS ON

CPU speed
compiler / language
cache and memory
other running processes

OPERATION COUNT FOCUSES ON

the algorithmic structure
input size
repeated work
input arrangement

OUR GOAL TODAY

Build a machine-independent mathematical description of work — before asymptotic notation.

Analysis abstracts away many hardware details so we can compare algorithms more fairly.

The simplified RAM model

An idealized machine model used to make operation counting manageable.

SEQUENTIAL

Instructions execute one after another.

→

UNIT-COST

Simple primitive operations are treated as constant-cost.

→

MEMORY ACCESS

Reading / writing a memory location is treated as constant-cost.

→

ABSTRACTION

Ignore CPU, cache, compiler and OS details.

The RAM model is a teaching abstraction — not a literal description of modern hardware.

What counts as a primitive operation?

For introductory analysis, we usually treat simple statements as constant-cost building blocks.

ASSIGNMENT

$x \leftarrow y$

ARITHMETIC

$x + y$, $x - y$, $x \times y$

COMPARISON

$x < y$, $x = y$

ARRAY ACCESS

$A[i]$

INCREMENT

$i \leftarrow i + 1$

RETURN

return x

We do not need to count every low-level CPU instruction to reason about algorithmic work.

STEP 1

First identify the input size

The symbol n must mean something concrete for the problem being analyzed.

ARRAY ALGORITHM

n = number of array elements

MATRIX ALGORITHM

n may be rows / columns or two dimensions m and n

STRING ALGORITHM

n = number of characters

Input size is part of the analysis — not an afterthought.

Always define n before writing T(n).

STEP 2

Then choose a basic operation

A basic operation is usually the operation whose repetition dominates the algorithm's work.

ALGORITHM

```
SUM(A)
    total ← 0
    for i ← 0 to n-1
        total ← total + A[i]
    return total
```

POSSIBLE BASIC OPERATION

total ← total + A[i] — the addition is executed once per element.

COUNT

For an array of n elements, the addition executes exactly n times.

$$C(n) = n$$

Pick an operation that reflects the repeated work you want to compare.

Basic-operation count vs. full step count

Both are valid models — but you must say which model you are using.

BASIC-OPERATION MODEL

Count one representative operation, such as the key comparison or addition.

Advantage: simple, highlights the main repetition.

Example: MAX comparisons = n − 1.

FULLER STEP MODEL

Count assignments, comparisons, increments, updates, etc.

Advantage: shows why input arrangement can change actual work.

Example: MAX updates vary with order.

Different cost models can produce different exact formulas — consistency matters.

EXAMPLE 1

Warm-up: analyze iterative summation

Start with an algorithm whose basic operation count is independent of the data values.

ALGORITHM

```
SUM(A)
    total ← 0
    for i ← 0 to n-1
        total ← total + A[i]
    return total
```

Operation	Count
total ← 0	1
addition total + A[i]	n
loop body executions	n
return	1

Basic additions: C(n) = n

The values in A do not change how many additions SUM performs.

Sometimes the input values matter — sometimes they do not

Do not assume every algorithm has different best and worst cases.

SUM(A)

Every element is added exactly once.

Same number of basic additions for every array of size n.

MAX(A)

Every candidate is compared, but the number of updates depends on order.

LINEAR SEARCH

Target position can drastically change the number of comparisons.

Case analysis makes sense only when the input characteristics can change the amount of work.

CASES

Best, worst, and average case

All three refer to inputs of the SAME size n.

BEST CASE

The least work among all valid inputs of size n.

WORST CASE

The most work among all valid inputs of size n.

AVERAGE CASE

Expected work under a stated probability model for inputs of size n.

Average case is meaningless unless the input distribution / assumption is stated.

BEST CASE

Best case ≠ “small input”

Best case means the easiest arrangement among inputs having the same size n.

LINEAR SEARCH EXAMPLE

A = [7, 2, 9, 5, 4]
search target v = 7

The target is at the first position.

BASIC COMPARISONS

Compare A[0] with v once.

Then return immediately.

$$C_{best}(n) = 1$$

n can be very large and the best case can still require only one key comparison.

WORST CASE

Worst case asks for the maximum work at size n

For linear search, this occurs when the target is last — or absent.

TARGET LAST

A = [7, 2, 9, 5, 4]
search v = 4

All earlier positions must be rejected first.

TARGET ABSENT

search v = 8

Every position is checked before NOT-FOUND can be returned.

$$C_{worst}(n) = n$$

Worst-case analysis provides a guaranteed upper workload for every input of size n.

Average case needs an assumption

Example: successful linear search with target equally likely to be in any of the n positions.

Possible comparisons: $1, 2, 3, \ldots, n$

Average: $(1 + 2 + \ldots + n) / n$

$= [n(n + 1)/2] / n$

$= (n + 1) / 2$

ASSUMPTION MATTERS

If target positions are not equally likely, the average changes.

Average-case analysis is always tied to a probability model.

Average successful linear-search comparisons: Cavg(n) = (n + 1)/2 under uniform target position.

WORKED EXAMPLE 2

Linear Search — full case comparison

A clean example where target position directly changes the work.

ALGORITHM

```
LINEAR-SEARCH(A, v)
    for i ← 0 to n-1
        if A[i] = v
            return i
    return NOT-FOUND
```

Case	Situation	Key comparisons
Best	v at A[0]	1
Worst	v last or absent	n
Average*	successful; uniform position	(n + 1)/2

*Average shown under an explicit assumption.

This is why “running time depends on input size” is incomplete — sometimes input arrangement matters too.

CDF EXAMPLE

Iterative MAX(A) — analyze the required example

We already proved it correct. Now we count its work.

ALGORITHM

```
MAX(A)
    largest ← A[0]
    for i ← 1 to n-1
        if A[i] > largest
            largest ← A[i]
    return largest
```

INPUT SIZE

n = number of elements in A

KEY COMPARISON

A[i] > largest

QUESTION

Does array order change the number of key comparisons?

MAX(A): key comparisons are fixed

After initializing with A[0], every remaining element is compared exactly once.

A[0]

used for initialization — no key comparison yet

→

A[1]

1 comparison

→

A[2]

1 comparison

→

… A[n−1]

1 comparison each

$$C_{comparison}(n) = n - 1$$

For the key comparison, best = worst = average = n − 1.

But input arrangement changes the number of updates

The CDF specifically asks us to analyze MAX under different input arrangements.

DESCENDING

A = [9, 7, 5, 3, 1]

largest starts at 9.
No later value is larger.

Updates to largest = 0

ASCENDING

A = [1, 3, 5, 7, 9]

Every new value is larger than the current largest.

Updates = n − 1

MIXED

A = [4, 9, 2, 11, 6]

Updates occur only when a new record maximum appears.

Updates = between 0 and n − 1

Comparisons stay fixed; assignment/update work can vary with arrangement.

Trace MAX on three arrangements

Same n. Same number of comparisons. Different number of updates.

Input	Comparisons	Updates to largest	Result
[9,7,5,3,1]	4	0	9
[1,3,5,7,9]	4	4	9
[4,9,2,11,6]	4	2	11

INTERPRETATION

If the basic operation is the comparison, all three inputs cost the same. If we also count assignments to largest, their exact work differs.

Always state what you are counting before claiming a best or worst case.

A fuller exact-count model for MAX(A)

We can separate the fixed work from the arrangement-dependent work.

Operation	Count
largest ← A[0]	1
key comparison A[i] > largest	n − 1
loop index increment	n − 1
largest ← A[i]	u, where 0 ≤ u ≤ n − 1
return	1

T(n, u) = fixed work + u

u = number of times a new record maximum appears.

Input characteristics can influence some operations even when the main comparison count is unchanged.

Loops become sums when the amount of work changes by iteration

Non-recursive does not mean “no mathematics.”

NESTED LOOP

```
for i ← 1 to n-1
    for j ← 0 to i-1
        do BASIC-OP
```

$$C(n) = 1 + 2 + 3 + \ldots + (n - 1)$$

$$= n(n - 1) / 2$$

English: iteration i executes the inner basic operation i times.

Today we derive exact counts. Lecture 7 will describe how these formulas grow.

SIGMA PREVIEW

A compact way to write repeated counts

Sigma notation is just shorthand for addition — understand the expanded sum first.

$$\sum_{i=1}^{n-1} i$$

means

$$1 + 2 + 3 + \ldots + (n - 1)$$

SYMBOLS

Σ = “sum”
i = running index
1 = starting value
n − 1 = ending value

CLOSED FORM

1 + 2 + … + (n − 1)
= n(n − 1)/2

We use sigma only as a counting tool here — asymptotic notation still waits for Lecture 7.

WORKED EXAMPLE 3

Count all unordered pairs in an array

A useful exact-count pattern appears in many brute-force algorithms later in the course.

ALGORITHM

```
for i ← 0 to n-2
    for j ← i+1 to n-1
        process pair (A[i], A[j])
```

i	Pairs processed
0	n − 1
1	n − 2
2	n − 3
…	…
n − 2	1

$$C(n) = (n - 1) + \ldots + 1 = n(n - 1)/2$$

Exact counts reveal structure that will later become complexity classes.

What input characteristics can affect performance?

The relevant characteristic depends on the algorithm.

TARGET LOCATION

Linear Search: first, middle, last, absent

ORDERING

MAX updates: ascending vs descending

DUPLICATES

Some algorithms may stop early or perform repeated work

VALUE DISTRIBUTION

Branch outcomes may change how often some statements execute

STRUCTURE / SHAPE

Later: trees and graphs can have very different shapes

INPUT SIZE

Almost every analysis starts by defining n

Do not talk about “the input” vaguely — identify which property actually changes the work.

Compare correct solutions using operation counts

Before Big-O, exact counting can already expose meaningful differences.

ITERATIVE SUM

```
S ← 0
for i ← 1 to n
    S ← S + i
```

Basic additions = n

DIRECT FORMULA

S ← n(n + 1) / 2

A fixed small number of arithmetic operations under the simplified RAM model.

Same required result → different operation counts

Correctness remains mandatory: an “optimization” is useful only if it preserves the required result.

CAUTION

Exact operation counts depend on the model

Do not pretend one formula is a universal physical runtime.

MODEL CHOICE

Is an array access one operation?
Do multiplication and addition cost the same?
Are assignments counted?

IMPLEMENTATION

Compiler and language can change actual machine instructions without changing the algorithm.

WHY ANALYSIS STILL HELPS

A consistent abstract model lets us reason about how work changes with input size and arrangement.

Analysis is a model of computational work, not a stopwatch reading.

METHOD

A systematic recipe for non-recursive analysis

Use the same sequence every time.

1 SIZE

Define n clearly.

→

2 BASIC OP

Choose the repeated operation.

→

3 COUNT

Find how often it executes.

→

4 CASES

Ask whether input arrangement changes the count.

→

5 FORMULA

Write C(n) or T(n).

Then interpret the result in plain English.

WORKED CHECK

Apply the recipe to FIND-MIN

Same structure as MAX — a good quick check of understanding.

ALGORITHM

```
MIN(A)
    smallest ← A[0]
    for i ← 1 to n-1
        if A[i] < smallest
            smallest ← A[i]
    return smallest
```

INPUT SIZE

n = number of array elements

BASIC OP

A[i] < smallest

COUNT

Exactly n − 1 key comparisons for every arrangement

The number of updates varies with order, just as in MAX.

CLASS PRACTICE

Your turn — analyze three non-recursive snippets

State n, choose a basic operation, and count it.

A COUNT EVEN

```
count ← 0
for i ← 0 to n-1
    if A[i] mod 2 = 0
        count ← count + 1
```

How many parity tests?

B FIND TARGET

```
for i ← 0 to n-1
    if A[i] = v
        return i
return NOT-FOUND
```

Give best / worst / average successful counts.

C PAIRS

```
for i ← 0 to n-2
    for j ← i+1 to n-1
        process pair
```

How many pairs are processed?

Do not write Big-O yet — give exact / concrete counts.

CHECK YOURSELF

Possible answers to the class practice

Equivalent reasoning is fine if the cost model is stated clearly.

A COUNT EVEN

n parity tests.

C(n) = n

The actual number of count increments depends on how many even values occur.

B FIND TARGET

Best = 1
Worst = n
Average successful = (n + 1)/2
under uniform target position.

C PAIRS

(n − 1) + (n − 2) + … + 1

= n(n − 1)/2 pair operations.

Separate fixed counts from data-dependent counts whenever useful.

Five analysis mistakes to avoid

Most errors happen before the algebra.

1. Never defining what n means.
2. Counting different operations in different algorithms without saying so.
3. Calling an input “best” because n is small.
4. Giving an average case without stating a probability assumption.
5. Jumping to Big-O before deriving or understanding the actual count.

Good analysis begins with a precise model, not with memorized complexity labels.

RECAP

Six ideas to carry forward

Lecture 6 gives us exact-count thinking; Lecture 7 turns it into growth-rate language.

01 RAM MODEL

idealized unit-cost analysis

02 INPUT SIZE

define n precisely

03 BASIC OPERATION

choose what repeated work to count

04 CASE ANALYSIS

best / worst / average depend on input

05 EXACT COUNTS

derive formulas such as $n - 1$ or $n(n - 1)/2$

06 SELECTION

compare correct solutions under one consistent model

Next: Asymptotic Analysis and Growth Rates — Big-O, Big-Ω, Θ and dominant growth.